

ECEN 428/722 - Lab Report

Lab Number: 1

Lab Title: CORDIC circuit design using VITIS HLS

Section Number: 602

Student's Name: Sai Vikhyat Pattar

Student's UIN: 936001854

Date: 9/11/2025

TA: Cristhian Roman Vicharra

Note: all sections listed below are mandatory in your post lab reports.

Objectives:

This lab introduces the Coordinate Rotation Digital Computer (CORDIC) algorithm and demonstrates its hardware implementation. CORDIC is an iterative approach designed to evaluate trigonometric, hyperbolic, and other mathematical functions like multiplication and division without using multipliers. It accomplishes this through repeated shift-and-add operations, making it highly suitable for hardware where multipliers are costly.

By systematically rotating vectors through predefined angles, the algorithm approximates the target function. To compute sine and cosine, the initial vector values are chosen as $x_0=1/G$, $y_0=0$, and z_0 as the input angle. After multiple iterations, the values converge to $\cos(z_0)$ and $\sin(z_0)$. This lab highlights the efficiency of CORDIC in real-world applications such as DSP, robotics, and embedded systems requiring fast and resource-conscious computations.

$$x_k = G(x_0 \cos z_0 - y_0 \sin z_0)$$

$$y_k = G(x_0 \sin z_0 + y_0 \cos z_0)$$

$$z_k = z_0 - \arctan(y_k/x_k)$$

Design:

The workflow for this lab consists of the following steps:

1. **Setup:** Download the provided CORDIC source code package and extract its contents.
2. **Project Creation:** Launch Vitis HLS, start a new project, and add the files (basic_cordic.cpp, basic_cordic.h, basic_cordic_tb.cpp).
3. **Board Selection:** Configure the target device as xc7z020clg400-1.
4. **Code Validation:** Resolve any syntax errors before proceeding.
5. **C Simulation:** Run a C simulation (Project → Run C Simulation). The included testbench validates the correctness of sine and cosine results against reference values, flagging discrepancies beyond 1%.
6. **C Synthesis:** Perform synthesis (Solution → Run C Synthesis → Active Solution), which translates the C implementation into RTL.
7. **C/RTL Co-Simulation:** Use the testbench to validate the RTL implementation. If successful, waveform verification confirms correctness.

Results:

Figure 1: C Simulation output for sine and cosine calculations.

```
18
19 theta_in = 1.04712
20 cos_ref = 0.5
21 cos_out = 0.499756
22 sin_ref = 0.866025
23 sin_out = 0.866455
24
25 theta_in = 0.497999
26 cos_ref = 0.766113
27 cos_out = 0.765625
28 sin_ref = 0.642378
29 sin_out = 0.642378
30
31 theta_in = 0.261719
32 cos_ref = 0.96592
33 cos_out = 0.96552
34 sin_ref = 0.259845
35 sin_out = 0.259521
36
37 theta_in = -0.349121
38 cos_ref = 0.939453
39 cos_out = 0.940918
40 sin_ref = -0.342285
41 sin_out = -0.343262
42
43 theta_in = -0.523482
44 cos_ref = 0.865947
45 cos_out = 0.865723
46 sin_ref = -0.500244
47 sin_out = -0.499756
48
49 theta_in = -0.959961
50 cos_ref = 0.573486
51 cos_out = 0.57373
52 sin_ref = -0.815336
53 sin_out = -0.815336
54
55 theta_in = -1.22192
56 cos_ref = 0.341797
57 cos_out = 0.341797
58 sin_ref = -0.939941
59 sin_out = -0.940186
60
61 theta_in = -1.53413
62 cos_ref = 0.0344238
63 cos_out = 0.0341797
64 sin_ref = -0.999012
65 sin_out = -0.999023
66
67 [Success] Your implementation of sine and cosine are correct!
68 INFO: [SIM 1] CSim done with 0 errors.
69 INFO: [SIM 3] ***** CSIM finish *****
70
```

Figure 2: RTL/C co-simulation results.

Cosimulation Report for 'cordic_sin_cos'

General Information

Date: Thu Sep 11 17:51:33 CDT 2025

Version: 2023.1 (Build 3854077 on May 4 2023)

Project: cordic_lab1

Status: Pass

Solution: solution1 (Vivado IP Flow Target)

Product family: zynq

Target device: xc7z020-clg400-1

Cosim Options

Tool: Vivado XSIM

RTL: Verilog

Dump Trace: all

Performance Estimates

Modules & Loops	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency
cordic_sin_cos 1	1	1	12	12	12	12

Figures 3.1, 3.2, and 3.3: Behavioral simulation outputs verifying CORDIC functionality.

Figure 3.1

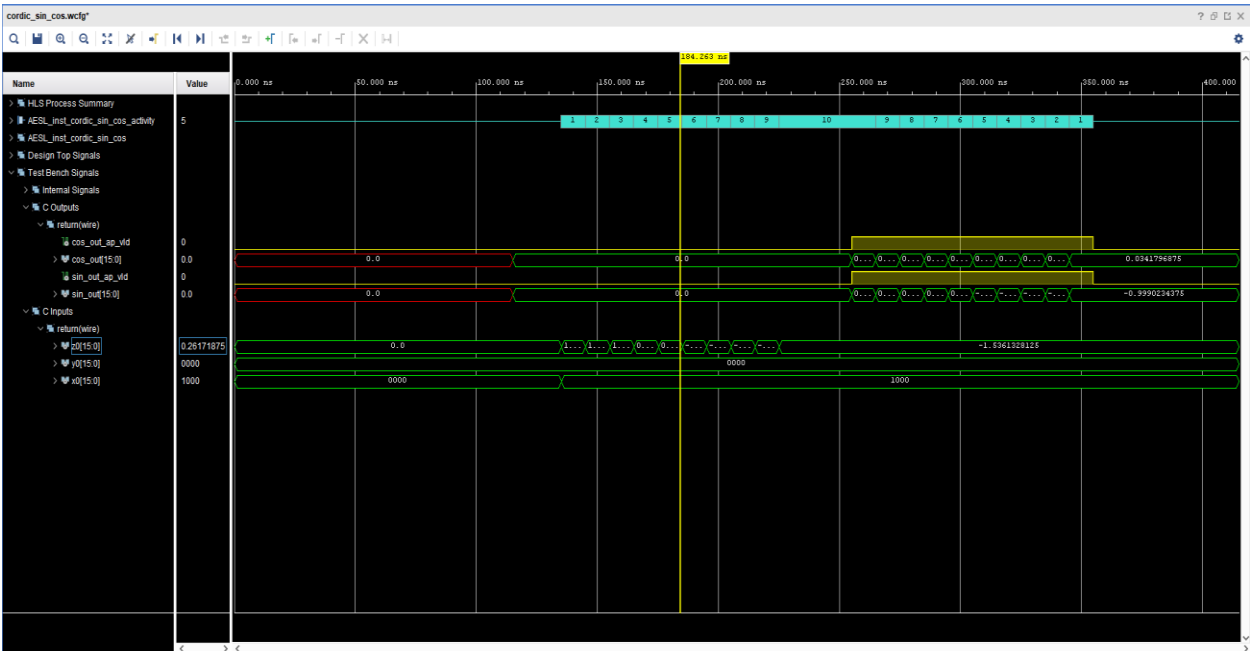


Figure 3.2

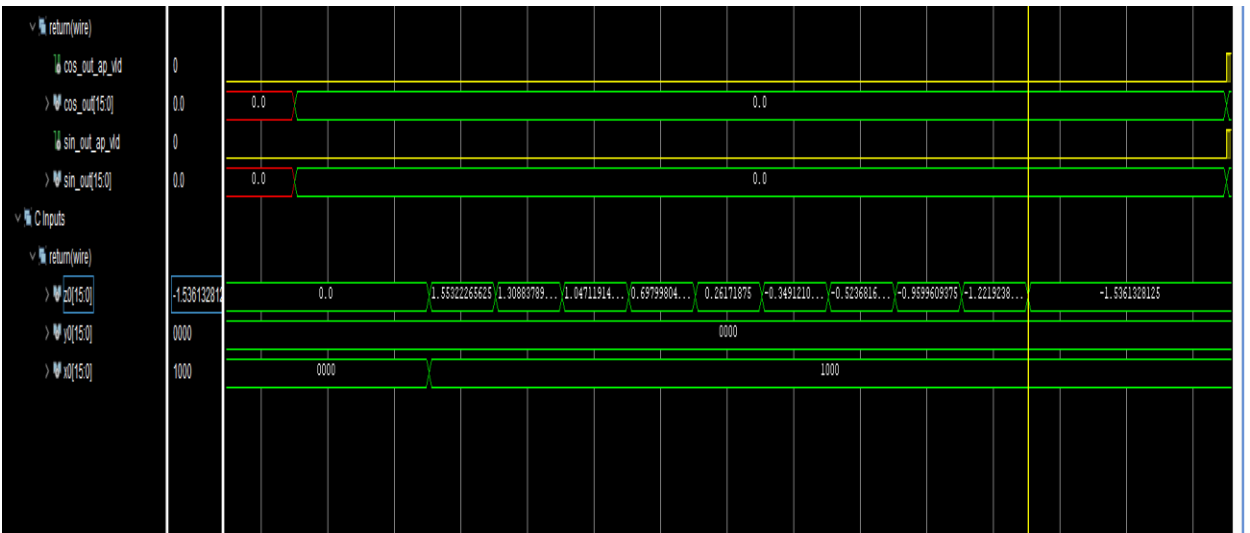
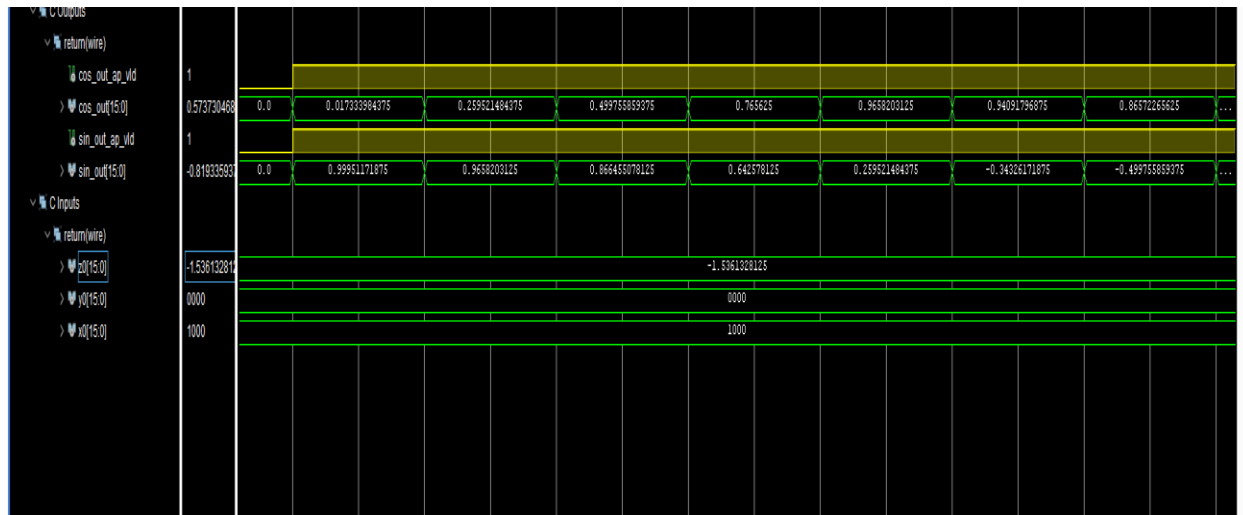


Figure 3.3



Conclusion:

In this lab, we explored how the CORDIC algorithm simplifies complex mathematical operations—such as trigonometric and hyperbolic functions—into iterative addition and shift operations. By tuning the number of iterations, a balance between accuracy and execution speed can be achieved, offering flexibility for different applications.

We demonstrated how initial conditions can be adjusted to compute sine, cosine, and inverse tangent values. Furthermore, slight modifications extend the algorithm to support hyperbolic functions. These features establish CORDIC as a highly effective and adaptable choice for digital hardware implementations in fields like DSP and embedded systems.